

Tutorato Programmazione 2

Michele Porcellato, Giacomo Vettore

6 Maggio 2026

1 Note

In caso di dubbi, consultare la *documentazione Java*. Sarà accessibile anche durante l'esame.

2 Teoria

Dovete valutare i seguenti codici ed indicare l'output oppure l'errore. In caso di errore, indicate a che riga è l'errore e, a grandi linee, di che tipo di errore si tratta.

```
1 import java.util.*;
2 public class Test {
3     public static void main(String[] args) {
4         String[] a = {"limone", "ananas", "mango", "lime"};
5         Set<Integer> s = new HashSet<>();
6         List<Integer> l = new ArrayList<>();
7         for(String i: a) {
8             s.add(i.length());
9             l.add(i.length());
10        }
11        System.out.println(s.size() + l.size());
12    }
13 }
```

```
1 public class Test {
2     public static void main(String[] args) {
3         I i = new C(4);
4         System.out.println(i.m(2));
5     }
6 }
7 interface I {
8     int m(int z);
9 }
10 class A implements I {
11     int x;
12     A(int x) { this.x = x + 1; }
13     public int m(int z) { return x + z; }
14 }
```

```

15 class B extends A {
16     B(int x) { super(++x); }
17     public int m(int z) { return x * z; }
18 }
19 class C extends B {
20     C(int x) { super(++x); }
21 }

```

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class Test {
5     public static void M(List<String> lista) {
6         System.out.println("Elaborazione lista di stringhe...");
7         for (String s : lista) {
8             System.out.println("- " + s);
9         }
10    }
11
12    public static void M(List<Integer> lista) {
13        System.out.println("Elaborazione lista di interi...");
14        for (Integer i : lista) {
15            System.out.println("- " + i);
16        }
17    }
18
19    public static void main(String[] args) {
20        List<String> A = new ArrayList<>();
21        A.add("Alice");
22        A.add("Bob");
23
24        List<Integer> B = new ArrayList<>();
25        B.add(10);
26        B.add(20);
27
28        M(A);
29    }
30 }

```

3 Esercizi

Si vuole realizzare l'applicazione "Ranked Games" che simula la gestione delle partite in un videogame multi-player online (es., BrawlStars). Attraverso l'interfaccia testuale, l'utente simula le azioni sia del giocatore che dell'avversario, nonché decide l'esito della partita.

Giocatore, rank, partita. Il giocatore ha associato un rank (grado) costituito da una categoria (Gold, Diamond, Master) e un livello (beginner, regular, expert) a seconda dell'esperienza maturata nelle partite giocate fino ad ora. Le partite sono caratterizzate dalle stesse tre categorie: un giocatore può giocare una partita soltanto se questa è di categoria pari o inferiore alla propria. Se la categoria è la stessa, in caso di vittoria il giocatore incrementa il proprio rank di un livello; viceversa, il rank non cambia in caso di sconfitta o di una partita di categoria inferiore.

Esempio: Si assuma che il giocatore abbia raggiunto rank pari Gold regular. Quando il giocatore vince una partita a livello Gold, il suo rank diventa Gold expert. Quando vince la partita successiva a livello Gold (l'unico possibile), il suo rank diventa Diamond beginner. A questo punto, il giocatore sceglie di giocare una partita a livello Diamond: se vince il suo rank diventa Diamond regular, mentre se perde (oppure decide di giocare invece una partita a livello Gold) il suo rank rimane inalterato.

La finestra base del programma (Esempio 1 / Esempio 1.1) mostra il rank (categoria e livello) del giocatore e il numero di punti esperienza (XP) accumulati finora. Inoltre, mostra tre bottoni, uno per ogni categoria di partita: soltanto quelli accessibili in base al rank del giocatore sono abilitati. All'avvio del programma, il rank del giocatore è Gold expert e i suoi XP sono pari a 1.000.000.

Personaggi e loro selezione. Il giocatore partecipa alla partita avvalendosi di una squadra di personaggi da lui scelti all'interno di quelli disponibili, ciascuno caratterizzato da un nome e da un livello numerico che ne rappresenta il valore. Alcuni dei personaggi, particolarmente rari, sono denominati Elite. Prima di iniziare una partita, il giocatore deve comporre la propria squadra, per combattere contro quella dell'avversario. Ciò avviene con modalità diverse a seconda della categoria della partita:

- **Gold.** Il giocatore seleziona due personaggi fra quelli a disposizione; questi possono essere costituiti anche da uno stesso personaggio selezionato due volte.
- **Diamond.** Prima della selezione del giocatore, l'avversario (simulato dallo stesso utente) può vietare l'utilizzo ("bannare") di alcuni personaggi, es., quelli più forti, disabilitandone esattamente due diversi fra loro. Terminata questa prima fase, il giocatore può selezionare i propri due personaggi da inserire nel team fra quelli rimanenti e non bannati; tuttavia, a differenza della categoria Gold, i personaggi selezionati devono essere diversi fra loro.
- **Master.** La procedura di ban e selezione è identica al caso Diamond: l'unica differenza è che la selezione è valida solo se effettuata su personaggi con un livello superiore a 10 e/o Elite.

L'Esempio 2 mostra un esempio in cui il giocatore, inserendo 1 nella finestra base, ha iniziato una partita Gold. Inizialmente, tutti i personaggi sono disponibili. I personaggi Elite sono denotati dal tag **[ELITE]**. Il giocatore di una partita Gold può scegliere lo stesso personaggio due volte. In caso il giocatore viene selezionato due volte, il tag sarà **[SELEZIONATO]**

L'Esempio 3 mostra invece un esempio in cui il giocatore ha scelto di giocare una partita Diamond. La selezione di un personaggio avviene cliccando sul quadrato corrispondente. L'avversario (simulato dall'utente) ha bannato due dei personaggi, alla quale si è aggiunto il tag **[BANNATO]**. Il giocatore (sempre simulato dall'utente) ha invece selezionato due dei personaggi rimanenti, il cui tag ora è **[SELEZIONATO]**.

Una volta che un personaggio è stato bannato oppure selezionato l'operazione non può essere cancellata. Si scrive a console un messaggio d'errore ogni qualvolta l'utente tenta di:

- bannare o selezionare più di due personaggi

- selezionare un personaggio bannato (solo in Diamond o Master)
- elezionare un personaggio che non è Elite né ha livello superiore a 10 (solo in Master)
- premere il bottone Vinci senza aver bannato e/o selezionato il numero richiesto di personaggi

Esito della partita. Una volta terminata la fase di selezione, l'utente simula l'esito della partita, determinando se il giocatore ha vinto oppure ha perso digitando v o p. In ambedue i casi viene mostrato stampato a video (Esempio 4) l'esito, seguito dal rank (categoria e livello), le caratteristiche dei personaggi che hanno partecipato alla partita, quelli eventualmente bannati e il totale di punti XP accumulati finora, pari a quelli precedenti alla partita sommati a quelli ottenuti in caso di vittoria. Quest'ultimo valore è determinato sommando i valori di XP generati da ciascuno dei personaggi impiegati:

- **Normale:** il numero di XP è pari a $100 \times L$, dove L è il livello del personaggio;
- **Elite:** il numero di XP è pari a quello normale a sua volta moltiplicato per $1000 \times C$, dove C dipende dalla categoria della partita e vale 1 per Gold, 2 per Diamond e 3 per Master.

Dopo di ciò si ritorna viene ripristinata la finestra base (Esempio 1.1) con il rank e numero di XP attuale.

Esempio 1

Giocatore: 1000000 XP, grado: GOLD (EXPERT)

Selezionare un livello con cui procedere:

1. GOLD

Livello:

Esempio 1.1

Giocatore: 1101100 XP, grado: MASTER (BEGINNER)

Selezionare un livello con cui procedere:

1. GOLD

2. DIAMOND

3. MASTER

Livello:

Esempio 2

Selezionare 2 giocatori:

1. PAM LV: 1

2. Piper LV: 1 [ELITE]

3. Kenji LV: 5 [ELITE]

4. Stecca LV: 5

5. Berry LV: 11

6. Jesse LV:11 [ELITE]

Giocatore 1: **1**

Selezionare 1 giocatori:

1. PAM LV: 1 [SELEZIONATO]

2. Piper LV: 1 [ELITE]

3. Kenji LV: 5 [ELITE]

4. Stecca LV: 5

5. Berry LV: 11

6. Jesse LV:11 [ELITE]

Giocatore 1: **1**

1. PAM LV: 1 [SELEZIONATO]

2. Piper LV: 1 [ELITE]

3. Kenji LV: 5 [ELITE]

4. Stecca LV: 5

5. Berry LV: 11

6. Jesse LV:11 [ELITE]

Vinci o perdi:

...

Esempio 3

Giocatore: 1101100 XP, grado: DIAMOND (BEGINNER)

Selezionare un livello con cui procedere:

1. GOLD
2. DIAMOND

Livello: 2

Selezionare 2 giocatori:

1. PAM LV: 1
2. Piper LV: 1 [ELITE] [BANNATO]
3. Kenji LV: 5 [ELITE] [SELEZIONATO]
4. Stecca LV: 5 [BANNATO]
5. Berry LV: 11 [SELEZIONATO]
6. Jesse LV:11 [ELITE]

Esempio 4

Hai Vinto.

Partita di livello: Diamond

Personaggi selezionati: Berry LV: 11 Kenji LV:5

Personaggi bannati: Piper LV: 1 Stecca LV: 5

Punti totalizzati 2102200

Requisiti

- Per testare le funzionalità si usino obbligatoriamente i dati visualizzati negli esempi. Tuttavia, il programma deve poter essere inizializzato con un numero arbitrario di personaggi.
 - Si usi, dove evidente/utile, una gerarchia di ereditarietà, specializzando opportunamente il comportamento delle classi e sfruttando ove possibile il polimorfismo.
 - Si usino costanti ove ragionevole (es., quando uno stesso valore viene usato più di una volta) e si badi alla pulizia del codice (es., linee guida Java) evitando duplicazioni e codice inutilmente complesso.
 - Al di fuori della definizione di costanti, si usi il modificatore static solo quando opportuno.
- i) Si consiglia di sviluppare prima il class diagram UML. È sufficiente la vista di alto livello (no attributi/metodi) ma devono essere mostrate le associazioni più importanti, oltre a quella di ereditarietà.